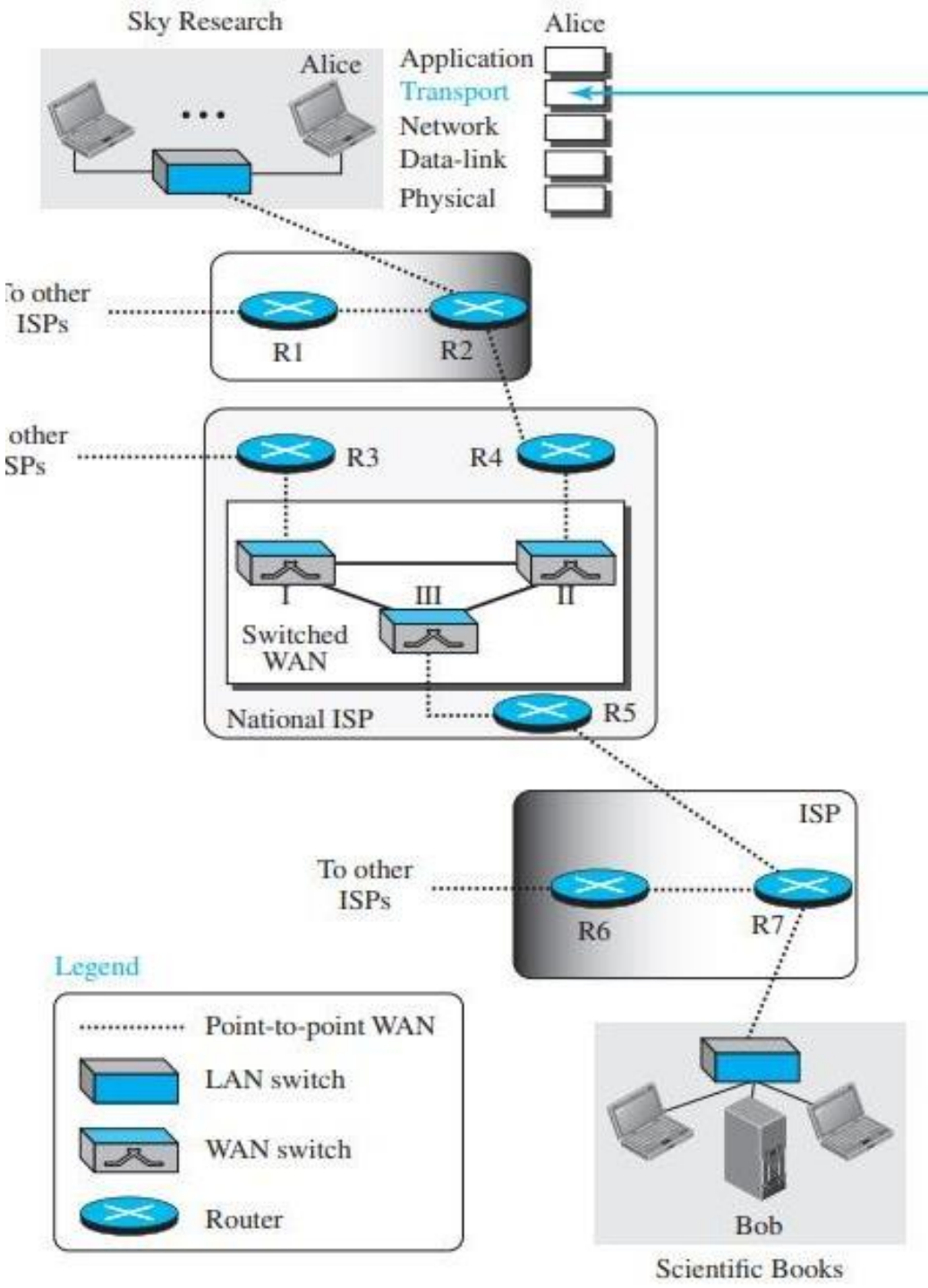
Data Communications and Networking Fourth Edition

Forouzan

Unit 3:Transport Layer

Transport Layer

- The transport layer is located between the application layer and the network layer.
- It provides a process-to- process communication between two application layers, one at the local host and the other at the remote host.
- Communication is provided using a logical connection, which means that the two application layers, which can be located in different parts of the globe, assume that there is an imaginary direct connection through which they can send and receive messages.



Transport layer services

1.PROCESS-TO-PROCESS *Communication*

The first duty of a transport-layer protocol is to provide process-to-process communication.

A process is an application-layer entity (running program) that uses the services of the transport layer.

The network layer is responsible for communication at the computer level (host-to-host communication).

A network-layer protocol can deliver the message only to the destination computer.

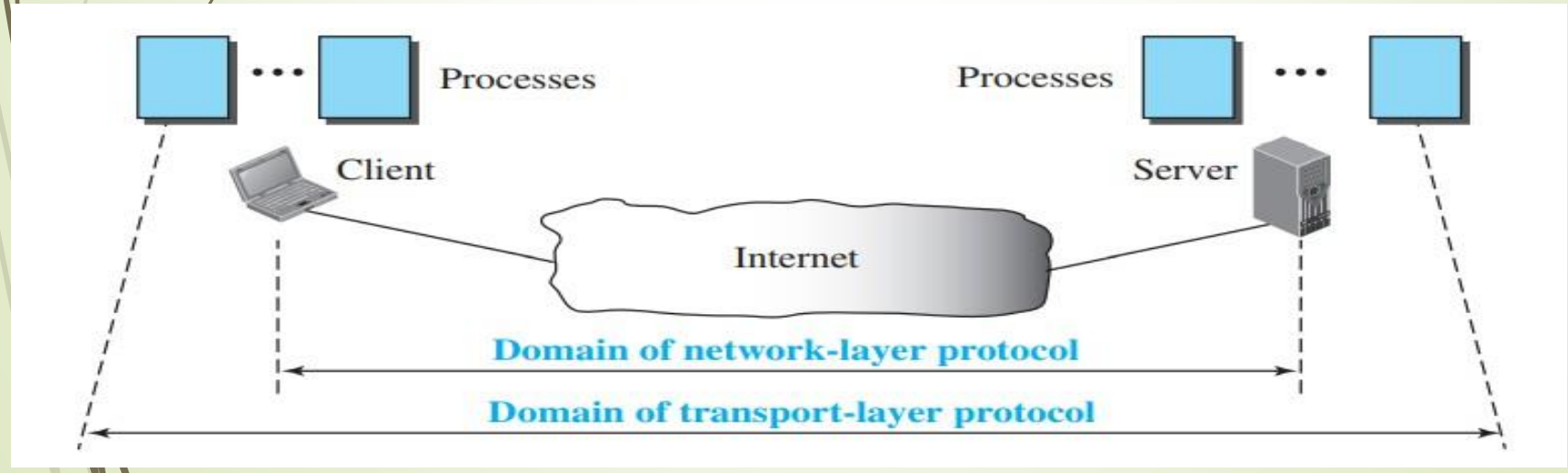
However, this is an incomplete delivery. The message still needs to be handed to the correct process.

This is where a transport-layer protocol takes over.

Transport layer services

1.PROCESS-TO-PROCESS *Communication*

A transport-layer protocol is responsible for delivery of the message to the appropriate process. Figure shows the domains of a network layer and a transport layer.



2. Addressing: Port Numbers

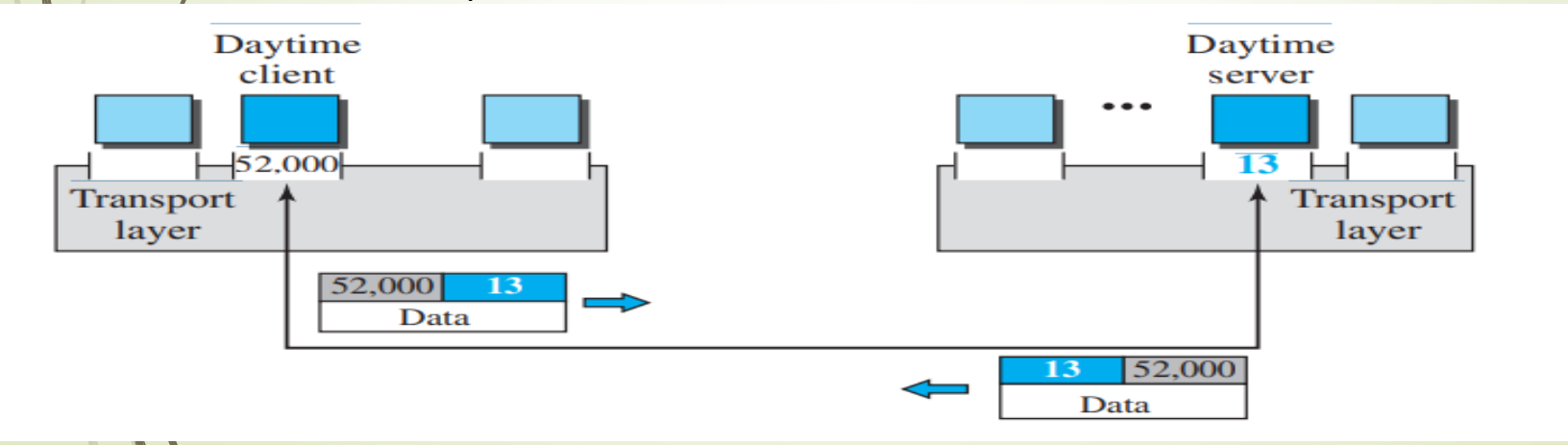
- Although there are a few ways to achieve process-to-process communication, the most common is through the client-server paradigm.
- A process on the local host, called a client, needs services from a process usually on the remote host, called a server.
- Operating systems today support both multiuser and multiprogramming environments.
- A remote computer can run several server programs at the same time, just as several local computers can run one or more client programs at the same time.
- For communication, we must define the local host, local process, remote host, and remote process.
- The local host and the remote host are defined using IP addresses.
- To define the processes, we need second identifiers, called port numbers.

2. Addressing: Port Numbers

In the TCP/IP protocol suite, the port numbers are integers between 0 and 65,535 (16 bits).
The client program defines itself with a port number, called the ephemeral port number.
The word ephemeral means "short-lived" and is used because the life of a client is normally short.
An ephemeral port number is recommended to be greater than 1023 for some client/server programs to work properly.
The server process must also define itself with a port number. This port number, however, cannot be chosen randomly.
If the computer at the server site runs a server process and assigns a random number as the port number, the process at the client site that wants to access that server and use its services will not know the port number.
One solution would be to send a special packet and request the port number of a specific server, but this creates more overhead.
TCP/IP has decided to use universal port numbers for servers; these are called well-known port numbers. There are some exceptions to this rule; for example, there are clients that are assigned well- known port

2. Addressing: Port Numbers

- TCP/IP has decided to use universal port numbers for servers; these are called well-known port numbers.
- There are some exceptions to this rule; for example, there are clients that are assigned well- known port numbers.
- Every client process knows the well-known port number of the corresponding server process. For example, while the daytime client process, a well-known client program, can use an ephemeral (temporary) port number, 52,000, to identify itself, the daytime server process must use the well-known (permanent) port number 13. Figure shows this concept.



Example of IP Address and port number

- **http://172.16.3.100:8090**

- This address contains several components, each with a specific meaning:
- ◈ 1. Protocol:**http://-** Specifies that the HyperText Transfer Protocol (HTTP) is used for communication. It tells the browser to use port 80 by default (unless another port is mentioned, as here).

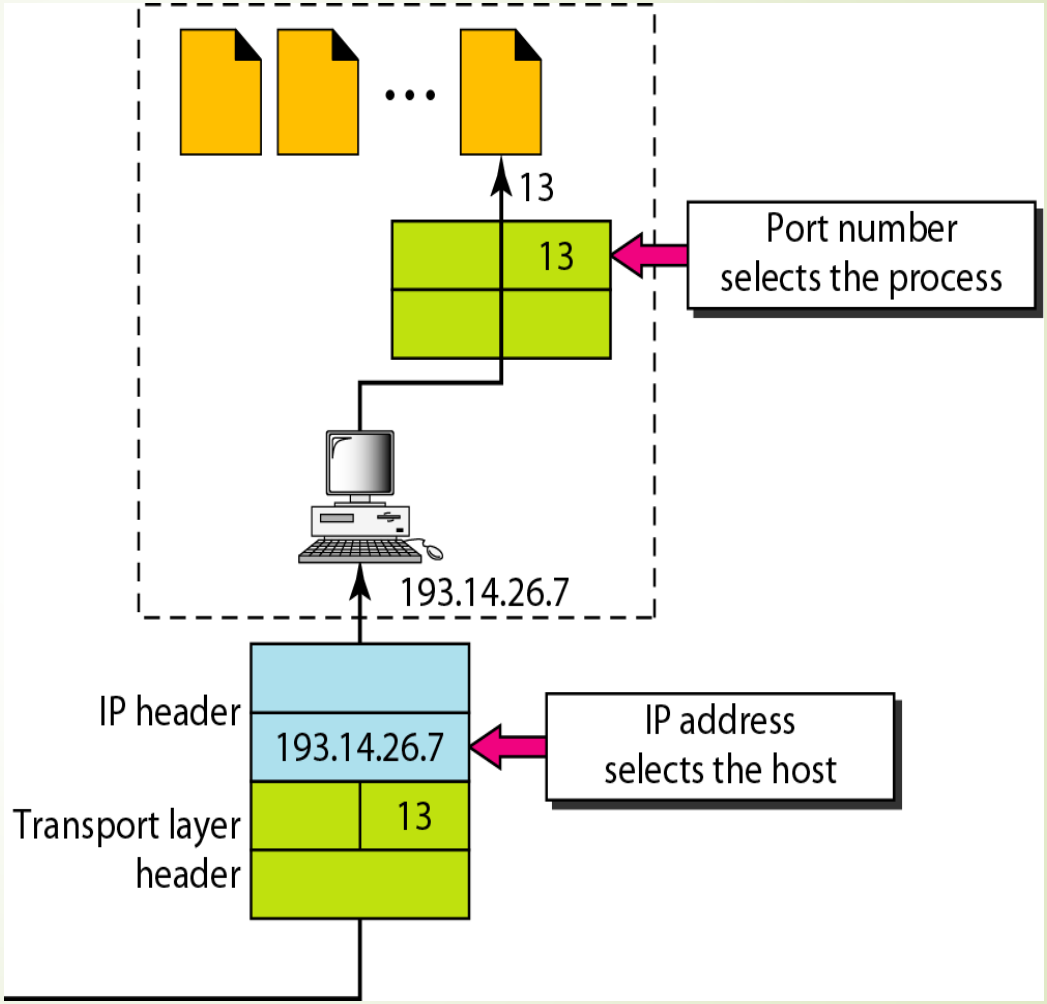
- ◈ 2. IP Address:**172.16.3.100-** This is the destination IP address — it identifies the host (computer/server) on the network where the resource resides.

- ◈ 3. Port Number:**:8090-** The number after the colon (:) indicates the port number.-
- Port 8090 is a non-standard port, often used when a web service or application server runs separately from the main web server (for example, Tomcat servers often use port 8080 or 8090).

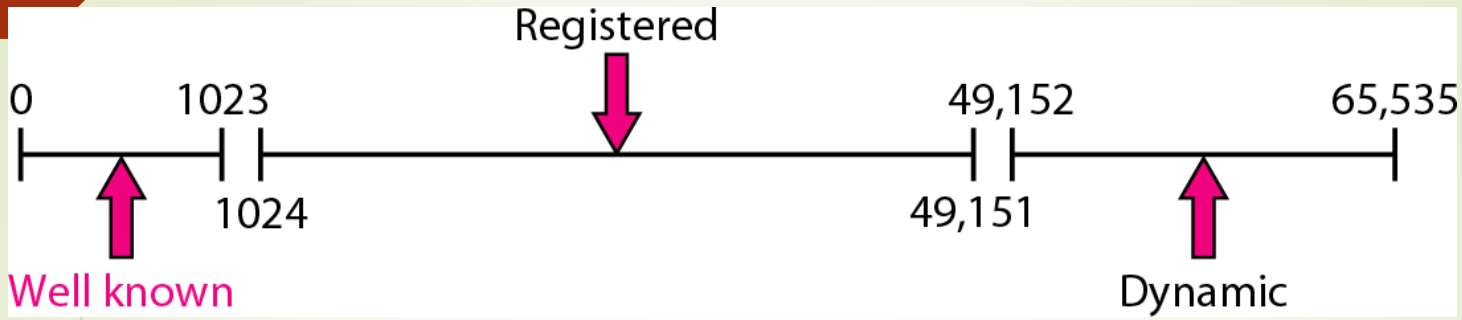
- It allows multiple services to run on the same IP by using different ports.

IP addresses versus port numbers

- The IP addresses and port numbers play different roles in selecting the final destination of data.
- The destination IP address defines the host among the different hosts in the world. After the host has been selected, the port number defines one of the processes on this particular host



ICANN ranges



ICANN has divided the port numbers into three ranges: well-known, registered, and dynamic (or private)

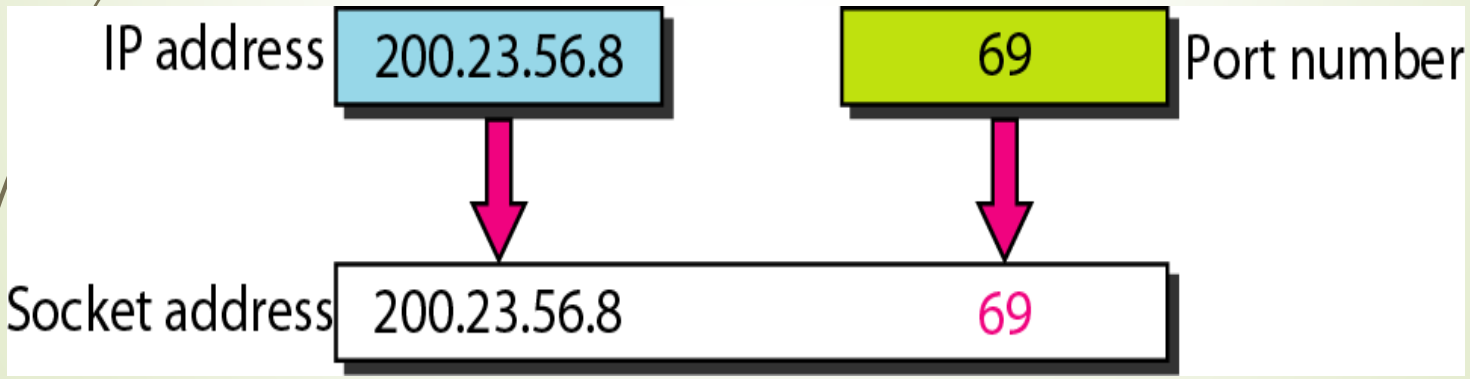
Well-known ports: The ports ranging from 0 to 1023 are assigned and controlled by ICANN. These are the well-known ports.

Registered ports: The ports ranging from 1024 to 49,151 are not assigned or controlled by ICANN. They can only be registered with ICANN to prevent duplication.

Dynamic ports: The ports ranging from 49,152 to 65,535 are neither controlled nor registered. They can be used as temporary or private port numbers.

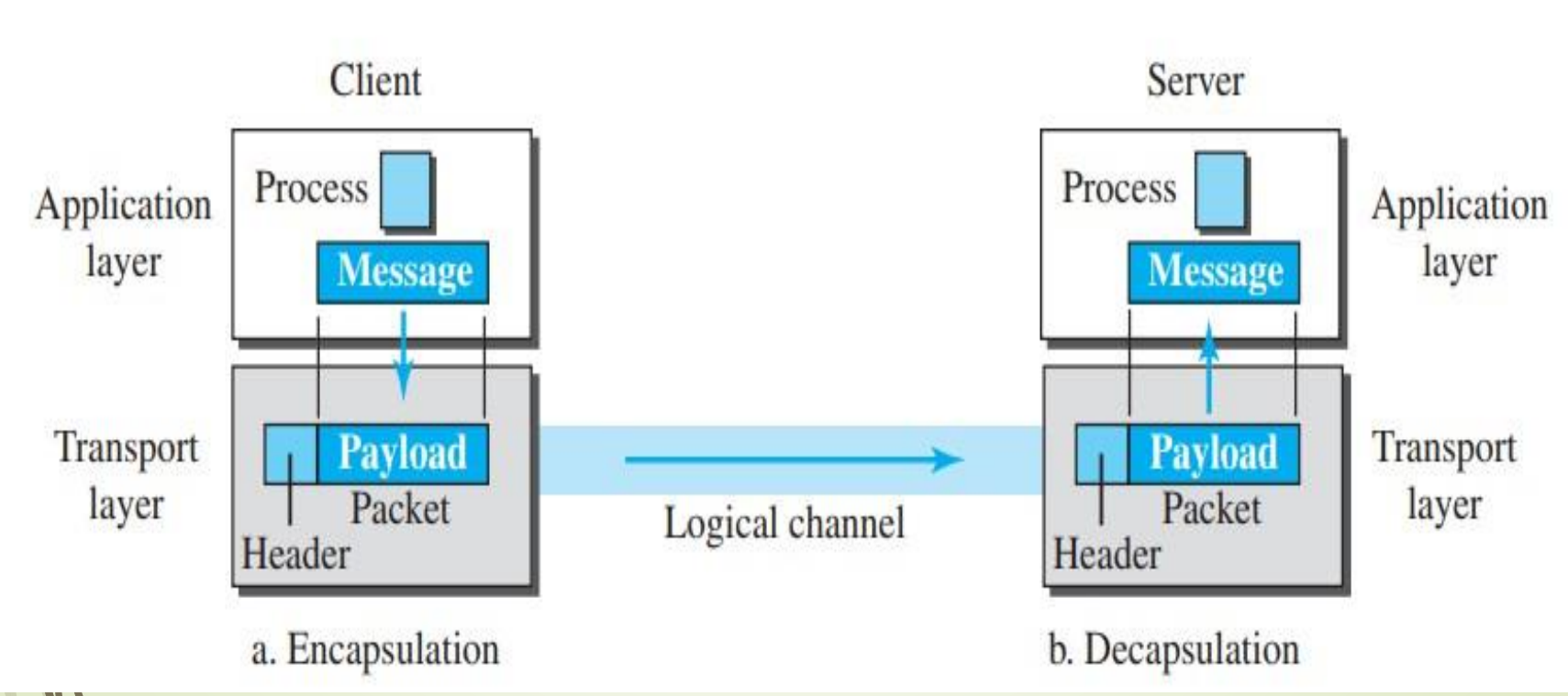
Socket address

- A transport-layer protocol in the TCP suite needs both the IP address and the port number, at each end, to make a connection.
- The combination of an IP address and a port number is called a socket address.
- The client socket address defines the client process uniquely just as the server socket address defines the server process uniquely .
- To use the services of the transport layer in the Internet, we need a pair of socket addresses: the client socket address and the server socket address.
- These four pieces of information are part of the network-layer packet header and the transport-layer packet header. The first header contains the IP addresses; the second header contains the port numbers.



3. Encapsulation and Decapsulation

1. To send a message from one process to another, the transport-layer protocol encapsulates and decapsulates messages as shown in below figure:



3. Encapsulation and Decapsulation

2. Encapsulation happens at the sender site. When a process has a message to send, it passes the message to the transport layer along with a pair of socket addresses and some other pieces of information, which depend on the transport-layer protocol.

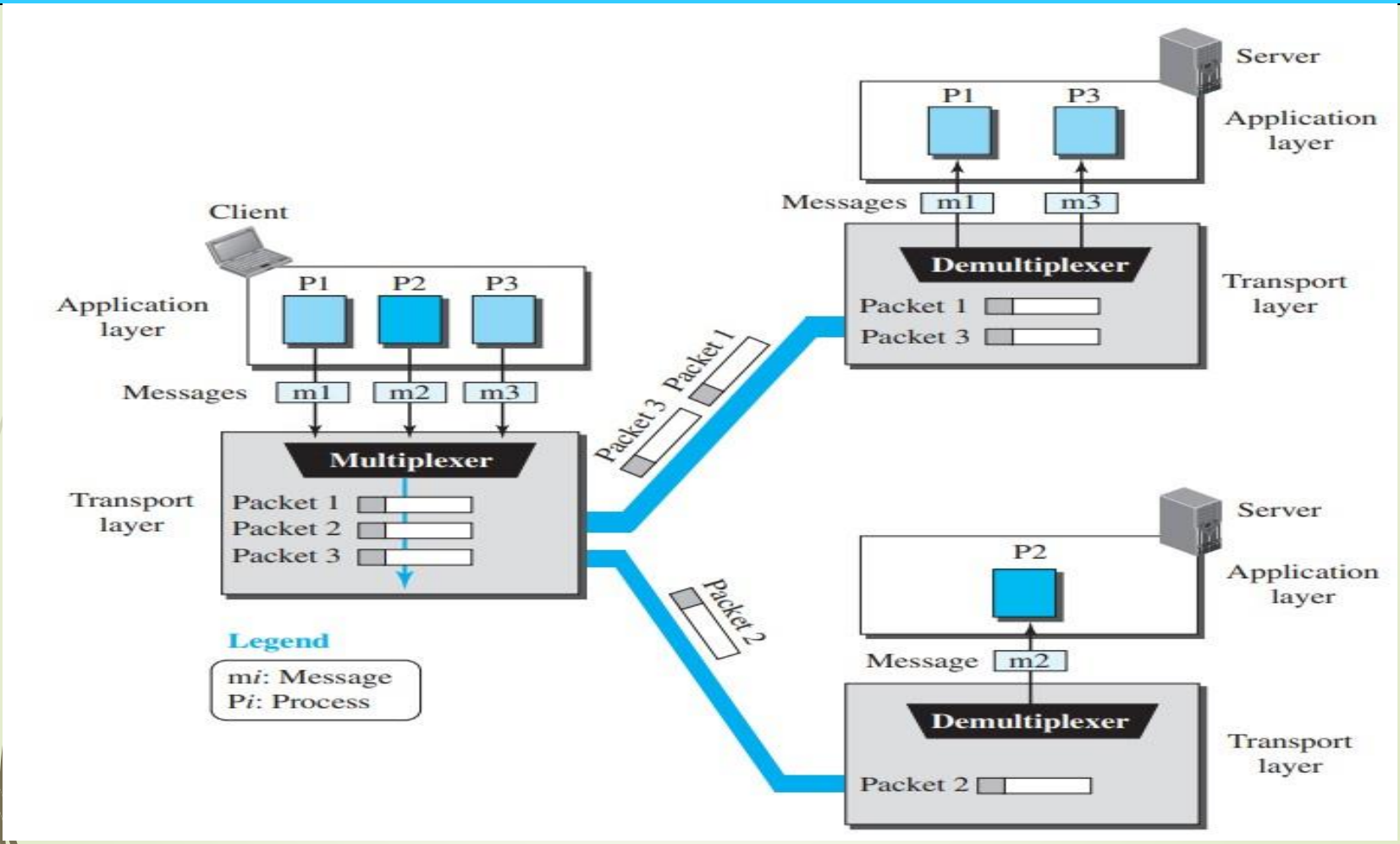
3. The transport layer receives the data and adds the transport-layer header. The packets at the transport layer in the Internet are called user datagrams, segments, or packets, depending on what transport-layer protocol we use.
In general discussion, we refer to transport-layer payloads as packets.

4. Decapsulation happens at the receiver site. When the message arrives at the destination transport layer, the header is dropped, and the transport layer delivers the message to the process running at the application layer. The sender socket address is passed to the process in case it needs to respond to the message received.

4. Multiplexing and Demultiplexing

a. Whenever an entity accepts items from more than one source, this is referred to as multiplexing (many to one); whenever an entity delivers items to more than one source, this is referred to as demultiplexing (one to many).
b. The transport layer at the source performs multiplexing; the transport layer at the destination performs demultiplexing. Figure shows communication between a client and two servers.
c. Three client processes are running at the client site, P1, P2, and P3. The processes P1 and P3 need to send requests to the corresponding server process running in a server.
d. The client process P2 needs to send a request to the corresponding server process running at another server. The transport layer at the client site accepts three messages from the three processes and creates three packets. It acts as a multiplexer.
e. The packets 1 and 3 use the same logical channel to reach the transport layer of the first server. When they arrive at the server, the transport layer does the job of a demultiplexer and distributes the messages to two different processes.
f. The transport layer at the second server receives packet 2 and delivers it to the corresponding process.

4. Multiplexing and Demultiplexing



5. Flow Control

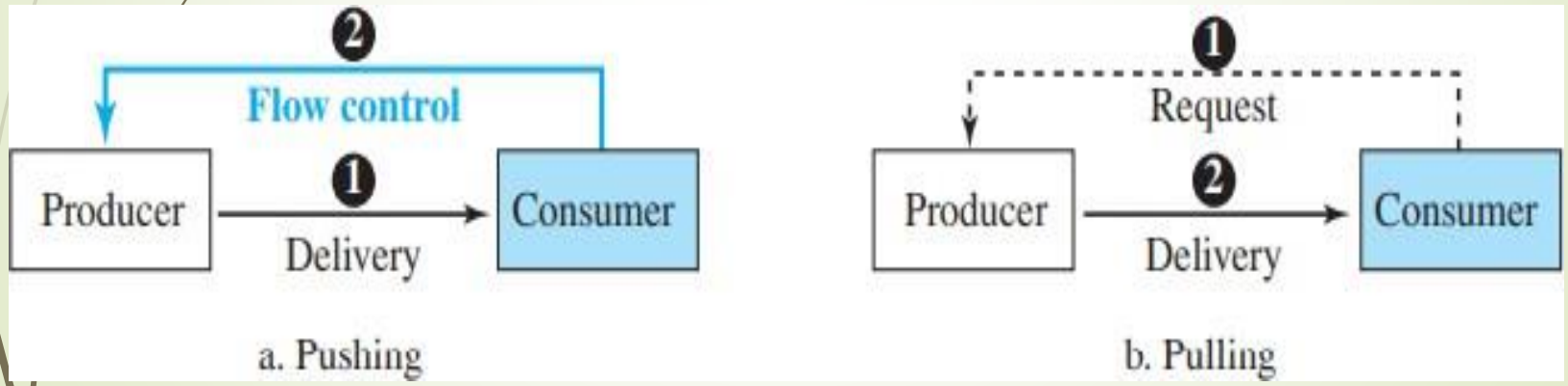
- Whenever an entity produces items and another entity consumes them, there should be a balance between production and consumption rates.
- If the items are produced faster than they can be consumed, the consumer can be overwhelmed and may need to discard some items.
- If the items are produced more slowly than they can be consumed, the consumer must wait, and the system becomes less efficient.

Pushing or Pulling:

- Delivery of items from a producer to a consumer can occur in one of two ways: pushing or pulling.
- If the sender delivers items whenever they are produced without a prior request from the consumer the delivery is referred to as pushing.
- If the producer delivers the items after the consumer has requested them, the delivery is referred to as pulling.

5. Flow Control

- When the producer pushes the items, the consumer may be overwhelmed and there is a need for flow control, in the opposite direction, to prevent discarding of the items.
- The consumer needs to warn the producer to stop the delivery and to inform the producer when it is again ready to receive the items.
- When the consumer pulls the items, it requests them when it is ready. In this case, there is no need for flow control.

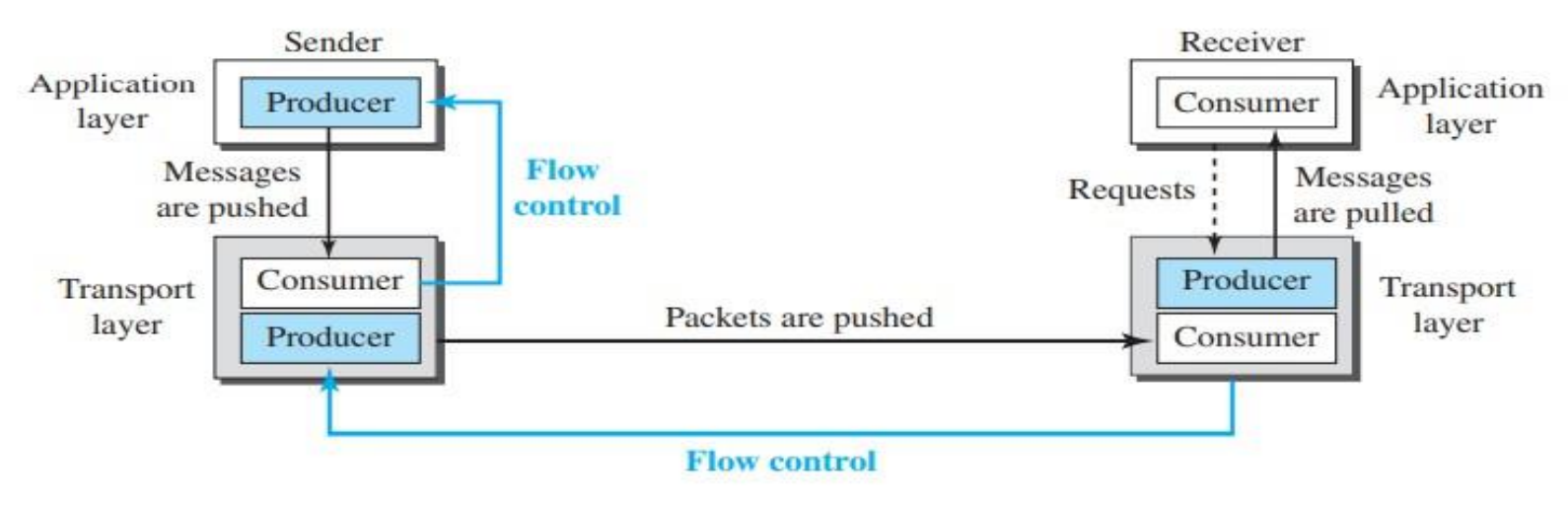


Flow Control at Transport Layer: :

In communication at the transport layer, we are dealing with four entities: sender process, sender transport layer, receiver transport layer, and receiver process.

The sending process at the application layer is only a producer. It produces message chunks and pushes them to the transport layer.

The sending transport layer has a double role: it is both a consumer and a producer. It consumes the messages pushed by the producer. It encapsulates the messages in packets and pushes them to the receiving transport layer.



Flow Control at Transport Layer:

The receiving transport layer also has a double role: it is the consumer for the packets received from the sender and the producer that decapsulates the messages and delivers them to the application layer.
The last delivery, however, is normally a pulling delivery; the transport layer waits until the application-layer process asks for messages.

Buffers

Although flow control can be implemented in several ways, one of the solutions is normally to use two buffers: one at the sending transport layer and the other at the receiving transport layer.
A buffer is a set of memory locations that can hold packets at the sender and receiver. The flow control communication can occur by sending signals from the consumer to the producer.
When the buffer of the sending transport layer is full, it informs the application layer to stop passing chunks of messages; when there are some vacancies, it informs the application layer that it can pass message chunks again.
When the buffer of the receiving transport layer is full, it informs the sending transport layer to stop sending packets. When there are some vacancies, it informs the sending transport layer that it can send packets again.

6. Error Control

In the Internet, since the underlying network layer (IP) is unreliable, we need to make the transport layer reliable if the application requires reliability.

Reliability can be achieved to add error control services to the transport layer.

Error control at the transport layer is responsible for detecting and discarding corrupted packets.

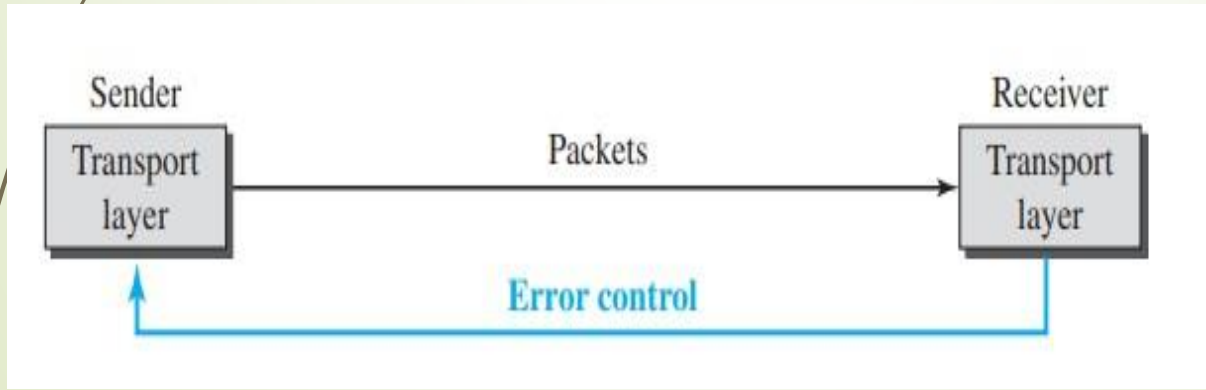
Keeping track of lost and discarded packets and resending them.

Recognizing duplicate packets and discarding them.

Buffering out-of-order packets until the missing packets arrive.

- **6. Error Control**

Error control, unlike flow control, involves only the sending and receiving transport layers.
As with the case of flow control, the receiving transport layer manages error control, most of the time, by informing the sending transport layer about the problems.



• Sequence Numbers

- Error control requires that the sending transport layer knows which packet is to be resent and the receiving transport layer knows which packet is a duplicate, or which packet has arrived out of order.
- This can be done if the packets are numbered. We can add a field to the transport-layer packet to hold the sequence number of the packet.
- When a packet is corrupted or lost, the receiving transport layer can somehow inform the sending transport layer to resend that packet using the sequence number.
- The receiving transport layer can also detect duplicate packets if two received packets have the same sequence number.
- The out-of-order packets can be recognized by observing gaps in the sequence numbers. Packets are numbered sequentially.Because we need to include the sequence number of each packet in the header, we need to set a limit.
- If the header of the packet allows m bits for the sequence number, the sequence numbers range from 0 to $2^m - 1$.
- For example, if m is 3, the only sequence numbers are 0 through 7, inclusive. However, we can wrap around the sequence. So, the sequence numbers in this case are 0, 1, 2, 3, 4, 5, 6, 7, 0, 1, 2, 3, 4, 5, 6, 7, In other words, the sequence numbers are modulo 2^m

- Acknowledgment:

a. We can use both positive and negative signals as error control, but we discuss only positive signals, which are more common at the transport layer.
b. The receiver side can send an acknowledgment (ACK) for each of a collection of packets that have arrived safe and sound.
c. The receiver can simply discard the corrupted packets. The sender can detect lost packets if it uses a timer.
d. When a packet is sent, the sender starts a timer. If an ACK does not arrive before the timer expires, the sender resends the packet.
e. Duplicate packets can be silently discarded by the receiver. Out-of-order packets can be either discarded (to be treated as lost packets by the sender) or stored until the missing one arrives.

- *Combination of Flow and Error Control:*

a. Flow control requires the use of two buffers, one at the sender site and the other at the receiver site.
b. Error control requires the use of sequence and acknowledgment numbers by both sides.
c. These two requirements can be combined if we use two numbered buffers, one at the sender, one at the receiver.
d. At the sender, when a packet is prepared to be sent, we use the number of the next free location, x, in the buffer as the sequence number of the packet.
e. When the packet is sent, a copy is stored at memory location x, awaiting the acknowledgment from the other end.
f. When an acknowledgment related to a sent packet arrives, the packet is purged, and the memory location becomes free.
g. At the receiver, when a packet with sequence number y arrives, it is stored at the memory location y until the application layer is ready to receive it. An acknowledgment can be sent to announce the arrival of packet y.

Example of sliding window

8. *Congestion Control*

An important issue in a packet-switched network, such as the Internet, is congestion.

- Congestion in a network may occur if the load on the network—the number of packets sent to the network—is greater than the capacity of the network—the number of packets a network can handle.
- Congestion control refers to the mechanisms and techniques that control the congestion and keep the load below the capacity. Congestion happens in any system that involves waiting.
- For example, congestion happens on a freeway because any abnormality in the flow, such as an accident during rush hour, creates blockage.
- Congestion in a network or internetwork occurs because routers and switches have queues—buffers that hold the packets before and after processing.
- A router, for example, has an input queue and an output queue for each interface. If a router cannot process the packets at the same rate at which they arrive, the queues become overloaded, and congestion occurs.
- Congestion at the transport layer is actually the result of congestion at the network layer, which manifests itself at the transport layer.

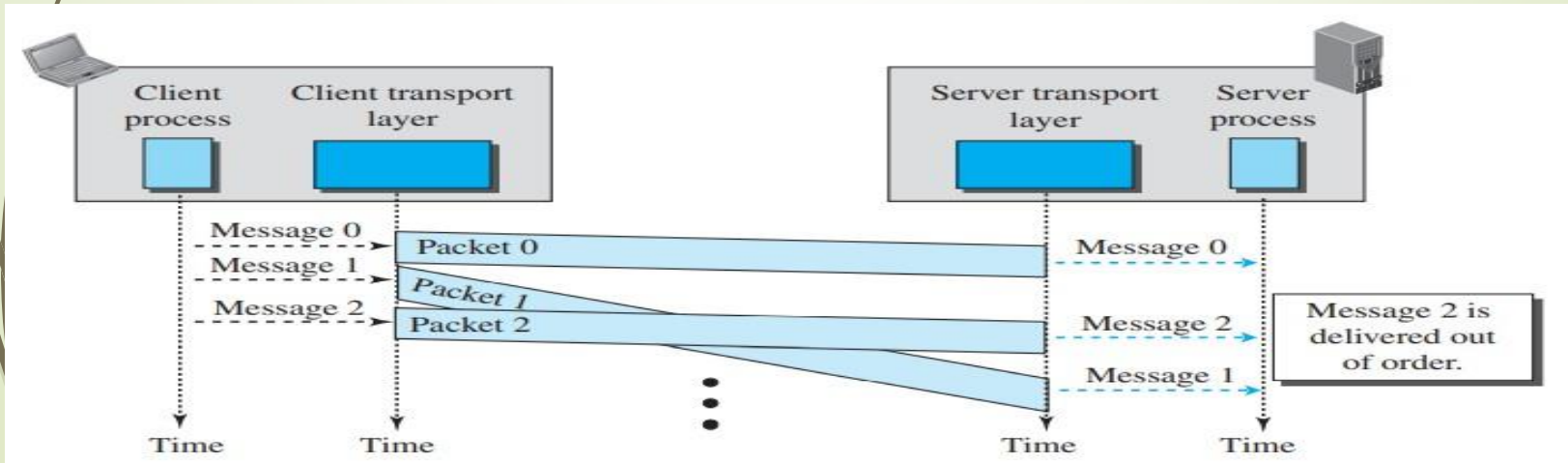
9. Connectionless and Connection-Oriented Protocols:

- A transport-layer protocol, like a network-layer protocol, can provide two types of services: connectionless and connection-oriented.
- The nature of these services at the transport layer, however, is different from the ones at the network layer.
- At the network layer, a connectionless service may mean different paths for different datagrams belonging to the same message.
- At the transport layer, we are not concerned about the physical paths of packets (we assume a logical connection between two transport layers).
- Connectionless service at the transport layer means independency between packets; connection-oriented means dependency

Connectionless Service

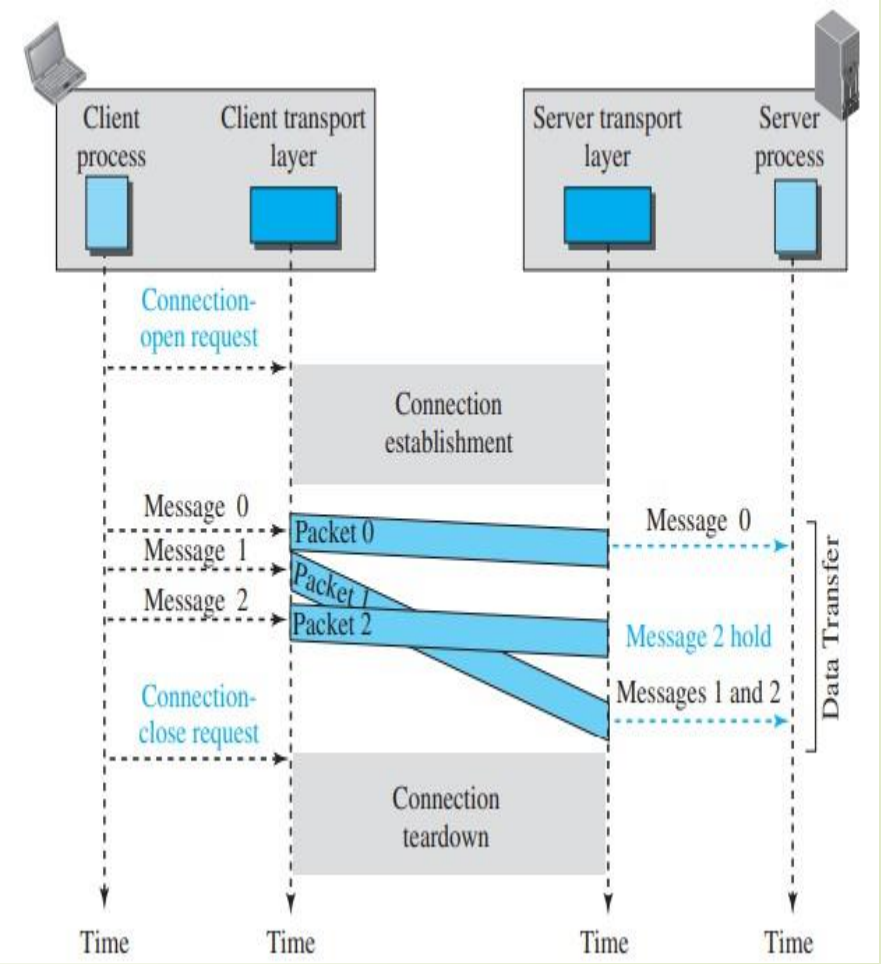
- In a connectionless service, the source process (application program) needs to divide its message into chunks of data of the size acceptable by the transport layer and deliver them to the transport layer one by one.
- The transport layer treats each chunk as a single unit without any relation between the chunks. When a chunk arrives from the application layer, the transport layer encapsulates it in a packet and sends it.
- To show the independency of packets, assume that a client process has three chunks of messages to send to a server process.
- The chunks are handed over to the connectionless transport protocol in order. However, since there is no dependency between the packets at the transport layer, the packets may arrive out of order at the destination and will be delivered out of order to the server process.

- In Figure, we have shown the movement of packets using a timeline, but we have assumed that the delivery of the process to the transport layer and vice versa are instantaneous.
- The figure shows that at the client site, the three chunks of messages are delivered to the client transport layer in order (0, 1, and 2). Because of the extra delay in transportation of the second packet, the delivery of messages at the server is not in order (0, 2, 1).
- If these three chunks of data belong to the same message, the server process may have received a strange message. The situation would be worse if one of the packets were lost.
- Since there is no numbering on the packets, the receiving transport layer has no idea that one of the messages has been lost. It just delivers two chunks of data to the server process.
- The above two problems arise from the fact that the two transport layers do not coordinate with each other. The receiving transport layer does not know when the first packet will come nor when all of the packets have arrived. We can say that no flow control, error control, or congestion control can be effectively implemented in a connectionless service.



Connection Oriented Service

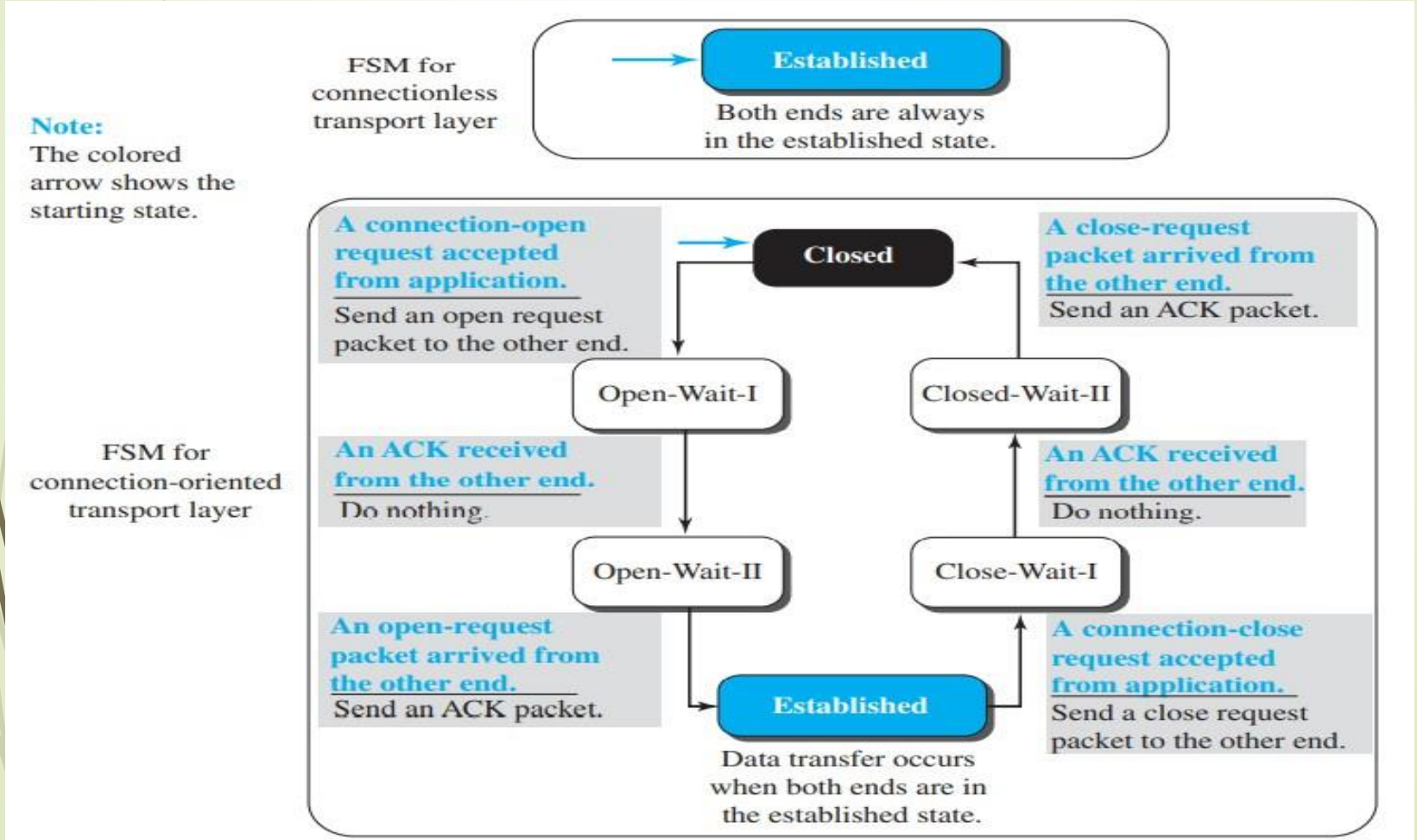
- In a connection-oriented service, the client and the server first need to establish a logical connection between themselves.
- The data exchange can only happen after the connection establishment. After data exchange, the connection needs to be torn down.
- The connection-oriented service at the transport layer is different from the same service at the network layer.
- In the network layer, connection-oriented service means a coordination between the two end hosts and all the routers in between.
- At the transport layer, connection-oriented service involves only the two hosts; the service is end to end. This means that we should be able to make a connection-oriented protocol at the transport layer over either a connectionless or connection-oriented protocol at the network layer.
- Figure shows the connection establishment, data-transfer, and tear-down phases in a connection-oriented service at the transport layer. We can implement flow control, error control, and congestion control in a connection-oriented protocol.



Finite State Machine:

- The behaviour of a transport-layer protocol, both when it provides a connectionless and when it provides a connection-oriented protocol, can be better shown as a finite state machine (FSM).
- Figure shows a representation of a transport layer using an FSM.
- Using this tool, each transport layer (sender or receiver) is taught as a machine with a finite number of states. The machine is always in one of the states until an event occurs.
- Each event is associated with two reactions: defining the list (possibly empty) of actions to be performed and determining the next state (which can be the same as the current state).
- One of the states must be defined as the initial state, the state in which the machine starts when it turns on.
- In this figure we have used rounded-corner rectangles to show states, coloured text to show events, and regular black text to show actions. A horizontal line is used to separate the event from the actions, although later we replace the horizontal line with a slash. The arrow shows the movement to the next state.
- We can think of a connectionless transport layer as an FSM with only one state: the established state. The machine on each end (client and server) is always in the established state, ready to send and receive transport-layer packets.

Finite State Machine:



Finite State Machine:

- An FSM in connection-oriented transport layer, needs to go through three states before reaching the established state.
- The machine also needs to go through three states before closing the connection. The machine is in the closed state when there is no connection. It remains in this state until a request for opening the connection arrives from the local process; the machine sends an open request packet to the remote transport layer and moves to the open-wait-I state.
- When an acknowledgment is received from the other end, the local FSM moves to the open-wait-II state. When the machine is in this state, a unidirectional connection has been established, but if a bidirectional connection is needed, the machine needs to wait in this state until the other end also requests a connection.
- When the request is received, the machine sends an acknowledgment and moves to the established state.

Finite State Machine:

- Data and data acknowledgment can be exchanged between the two ends when they are both in the established state.
- However, we need to remember that the established state, both in connectionless and connection- oriented transport layers, represents a set of data transfer states.
- To tear down a connection, the application layer sends a close request message to its local transport layer.
- The transport layer sends a close-request packet to the other end and moves to close-wait-I state. When an acknowledgment is received from the other end, the machine moves to the close-wait-II state and waits for the close-request packet from the other end.
- When this packet arrives, the machine sends an acknowledgment and moves to the closed state. There are several variations of the connection-oriented FSM.

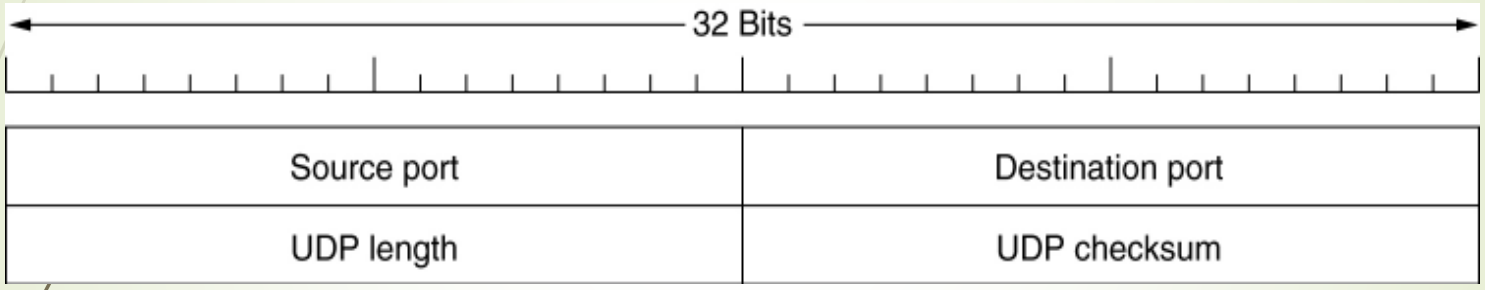
Transport layer Protocols:

- UDP
- TCP
- SCTP

Introduction to UDP

- The Internet protocol suite supports a connectionless transport protocol, UDP (User Datagram Protocol).
- UDP provides a way for applications to send encapsulated IP datagrams and send them without having to establish a connection.
- UDP transmits segments consisting of an 8- byte header followed by the payload.
- The two ports serve to identify the end points within the source and destination machines.
- When a UDP packet arrives, its payload is handed to the process attached to the destination port.

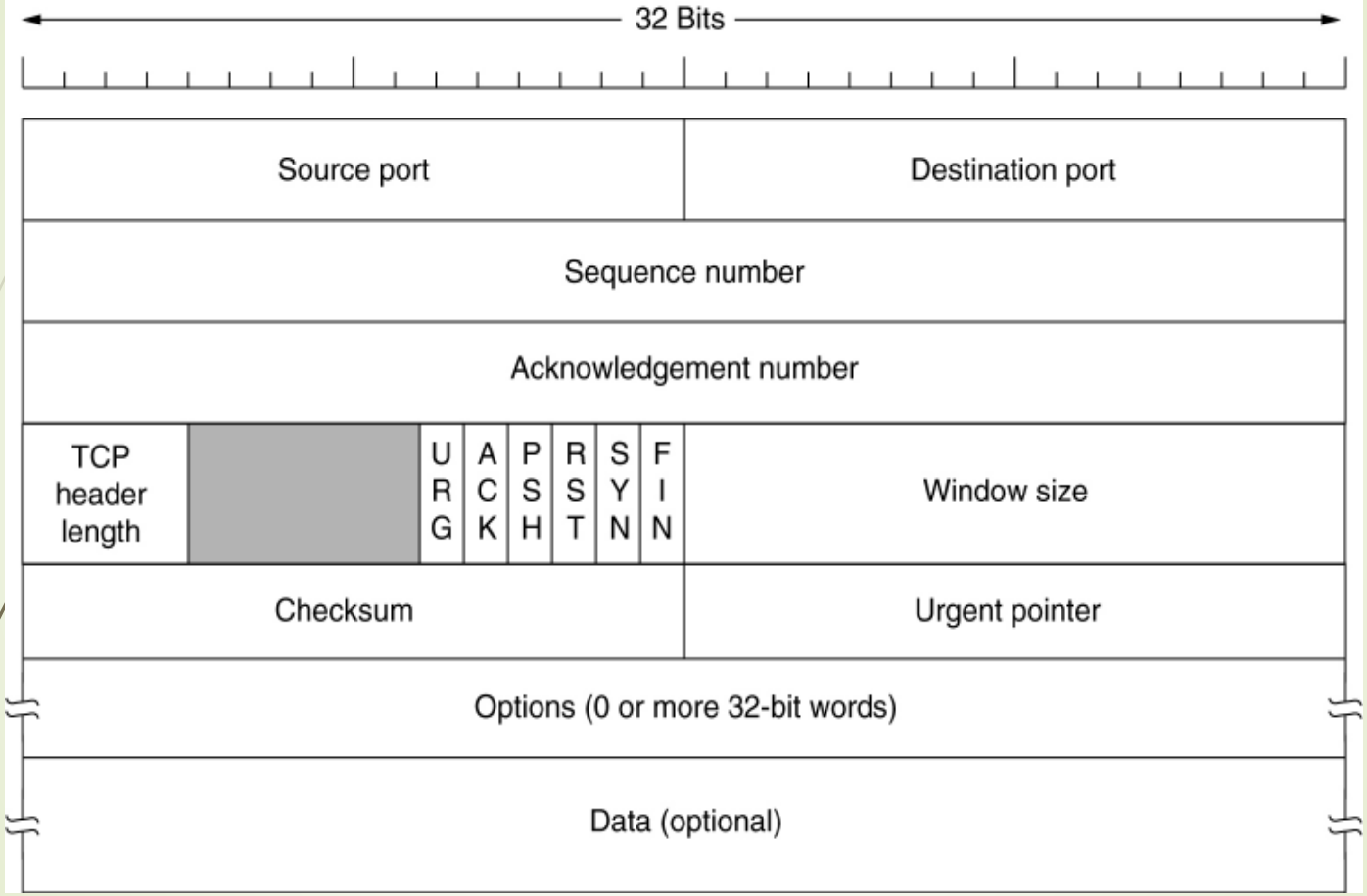
UDP header


32 Bits
Source port
Destination port
UDP length
UDP checksum

The TCP Segment Header

- Every segment begins with a fixed-format, 20-byte header.
- The fixed header may be followed by header options.
- After the options, if any, up to 65,535-20- 20=65,495 data bytes may follow, where the first 20 refer to the IP header and the second to the TCP header.
- Segments without any data are legal and are commonly used for acknowledgements and control messages.
- Source port and destination port fields identify the local end points of the connection.
- The source and destination end points together identify the connection.

The TCP Segment Header



TCP Header.

Source Port: It is a 16-bit source port number used by the receiver to reply.

Destination Port: It is a 16-bit destination port number.

Sequence Number: Specifies sequence number of the data .

Acknowledgement Number: If the ACK control bit is set, this field contains the next number that the receiver expects to receive.

TCP header length:Tells how many 32 bit words are contained in TCP header.

6 bit fields -not used

URG:It indicates an urgent pointer field that data type is urgent or not. Set to 1 if urgent pointer is in use.

ACK: It indicates that the acknowledgement field in a segment. Set to 1 to indicate that the acknowledgement number is valid. If 0, then it indicates that segment does not contain any acknowledgement.

PUSH: The PUSH flag is set or reset according to a data type that is sent immediately or not. The receiver is hereby requested deliver the data to the application upon arrival not to buffer it untill a full buffer has been received.

.

RST: It Resets the connection that has become confused due to host crash or some other reason.

SYN: It synchronizes the sequence number.Used to denote the connection request and connection accepted.

FIN: This indicates no more data from the sender.It is used to release the connection
Window: It is used in Acknowledgement segment for flow control. It specifies the number of data bytes, beginning with the one indicated in the acknowledgement number field that the receiver is ready to accept.

Checksum: It is used for error detection.

Options: This field provides a way to add extra facilities not covered by the regular header.

Data: Although in some cases like acknowledgement segments with no data in the reverse direction, the variable-length field carries the application data from sender to receiver. This field, connected with the TCP header fields, constitute a TCP segment.

Urgent pointer : This pointer specifies the position where urgent data ends.